

Agentic Prescriptive Analytics for Financial Services — From Prediction to Action

A code-verified mapping of the Prediction -> Insight -> Recommendation -> Optimization -> Governance -> Action -> Outcome journey onto the actual application — prepared for live Hackfest demonstration and judge review.

Generated September 19, 2026

Table of Contents

- 1. Executive Summary
- 2. Business Problem
 - What happens in a traditional prediction-only system
 - Why prediction/scoring alone is insufficient
 - How this application converts a signal into a governed action
 - Where AI helps, where rules are used, where optimization is used, where humans approve
 - The requested journey, mapped honestly
- 3. Solution Overview
- 4. Application Overview
- 5. User Roles
- 6. Application Modules
- 7. End-to-End Architecture
- 8. Architecture Diagram
- 9. Agentic AI Architecture
 - What "agent" means in this codebase
 - The four real agents
 - Agent flow (generic shape, applies to all four)
- 10. LLM Integration
- 11. Deterministic Engines
- 12. Data Architecture
- 13. API / Backend Flow
- 14. Security & Governance
- 15. End-to-End Business Flow
- 16. Prediction -> Action Journey
- 17. Implemented vs Proposed Features

- 18. Hackfest Demo Flow (10–15 minutes)
- 19. Detailed Presenter Script
- 20. Architecture Walkthrough Script (2–3 minutes)
- 21. Business Value
- 22. Technical Innovation
- 23. Future Enhancements
- 24–25. Judge Questions and Answers
- 26. One-Minute Elevator Pitch
- 27. Three-Minute Technical Pitch
- 28. Final Demo Checklist
- Hackfest Story (closing)

Scope note (read first): This document analyzes **Zuci Agent Services** (repository [insurance-fax-app](#)), the only application available for this analysis — an insurance claim-intake and AI-transformation platform, not a banking/credit-collections system. The Hackfest theme ("Agentic Prescriptive Analytics for Banking") is applied to this application honestly: every stage of the requested Prediction -> Insight -> Recommendation -> Optimization -> Governance -> Action -> Outcome journey is mapped to a **real** feature where one exists, and explicitly marked **"Not identified in the current implementation"** or **"Proposed / Future Enhancement"** where it does not. Nothing below is invented to fit the theme. Where the application's own domain language (insurance claims) differs from the Hackfest's banking framing, this document says so rather than silently relabeling one as the other.

1. Executive Summary

Application: Zuci Agent Services — an insurance claim-intake platform that extracts, verifies, and routes faxed/scanned claim documents, paired with a "Growth Studio" consulting toolkit that assesses *any* business process for agent-automation fit.

What this Hackfest story is really about: most "AI transformation" demos show a prediction and stop. This application is used here to demonstrate the *next* step — turning an existing signal (a document's extraction confidence and relevance score) into a **governed, explainable, auditable action**, with a human kept in the loop exactly where the system is not confident enough to act alone. The same governance pattern — *the model drafts, deterministic code decides* — is applied consistently across four separate AI touch points in the codebase, which is the strongest, most repeatable evidence for the "agentic prescriptive analytics" narrative this application can honestly support.

What is genuinely implemented:

- A real, working extraction pipeline (PDF -> OCR fallback -> AI/rule-based field extraction -> source verification -> conflict detection)
- A real, code-enforced policy engine (three configurable thresholds that decide auto-process / human-required / reject)
- A real, code-enforced hallucination guardrail on every LLM call (whitelist filtering, source-text grounding checks, confidence-based overrides)
- A real audit trail and live governance/operations views over the same claim data

What is honestly not implemented (and must not be presented as if it were):

- No trained prediction/ML model (no forecasting, no churn/risk model, no scikit-learn/PyTorch dependency)
- No optimization solver (no MIP/CBC, no alternative-comparison engine) — ROI/Revenue figures are plain arithmetic, not optimization
- No multi-role access model (one mock role, `Clinical Admin`, exists in the whole application)
- No backend authentication (the login screen gates the Angular app only, not the API)
- No persistence (all data is in-memory Python dictionaries, lost on backend restart)

Elevator framing: *"We are not pitching a prediction model. We are showing the layer that comes after a prediction exists — the deterministic, policy-governed action layer that a real AI-in-banking rollout actually needs, built and proven on a working application, not slides."*

2. Business Problem

What happens in a traditional prediction-only system

A model (or, as here, a deterministic scoring step) produces a number — a confidence score, a match score, a risk grade. In most demos, the story stops there: the number is shown on a dashboard, and a human has to figure out, unaided, what to actually

do

about it, using judgment that is not recorded, not consistent between reviewers, and not auditable.

Why prediction/scoring alone is insufficient

- A confidence score is not a decision — someone still has to decide whether "72% confident" is good enough to act on.
- Without a codified policy, two reviewers can reach different conclusions from the same score.
- Without an audit trail, nobody can later explain *why* a specific document was auto-processed while another, similar one was escalated.
- Without a guardrail on the AI itself, there is no defense against the model inventing a plausible-looking but false value.

How this application converts a signal into a governed action

1. **Signal:** every uploaded document gets a deterministic relevance/match score (`DocumentRelevanceService`) and, per field, an AI-or-rule-based extraction confidence.
2. **Policy:** three configured thresholds (`invalid_match_threshold=25` , `review_match_threshold=60` , `low_confidence_threshold=0.70` , from `backend/app/core/config.py`) turn that signal into one of exactly three outcomes — reject, human-required, or straight-through — with **no other code path** to a different outcome.
3. **Human gate:** anything that does not clear the thresholds is routed to a reviewer in the Claim Queue, who can ask the grounded AI Claim Manager questions before deciding, and must explicitly "Approve & Resolve."
4. **Audit:** every state-changing action (upload, decision, correction, delete, restore, reprocess) is appended to an in-memory audit log, immediately visible on the Governance and Managed Operations pages.

Where AI helps, where rules are used, where optimization is used, where humans approve

Journey stage	What actually does the work	Real / Not identified
Producing the signal	AI (Gemini) extraction confidence + a deterministic keyword-based relevance scorer	Real
Deciding the action from the signal	Deterministic threshold comparison (<code>core/config.py</code> values)	Real
Drafting a recommendation narrative	AI (Gemini), JSON-mode, per feature	Real
Enforcing what the AI is allowed to recommend	Deterministic whitelist filter (Discovery agent)	Real
Comparing alternative actions to pick the best one	—	Not identified in the current implementation
Approving the final decision	A human, via "Approve & Resolve"	Real
Recording what happened and why	In-memory audit log + Governance page	Real (not persisted across restarts)

The requested journey, mapped honestly

Prediction	-> Not identified as a trained ML model. The closest real analog is the document relevance/match score and the AI extraction's own per-field confidence -- both are point-in-time signals computed on the document just uploaded, not a forecast of a future event.
Insight	-> Real. Governance and Managed Operations pages show live, descriptive analytics (decision-authority split, confidence distribution, audit trail) computed from actual claim data.
Recommended Action	-> Real. The claim is routed to exactly one of three outcomes by policy; Growth Studio's Discovery agent additionally recommends an agent architecture for a *described process* (a separate, consulting-style workflow, not the claim pipeline itself).
Optimization	-> Not identified in the current implementation. ROI/Revenue calculators are deterministic arithmetic over user-entered inputs, not a solver comparing alternative actions. Proposed / Future Enhancement.
Governance / Policy	-> Real. Three configured thresholds, enforced in code, on every document,

Validation	every time, with no bypass. Visible live on the Governance page.
Human Approval	-> Real, where the policy requires it. "Review Anyway" gate for borderline documents; "Approve & Resolve" is always a human action.
Action	-> Real. Claim status is set (auto_filled / needs_review / resolved / invalid); this is the actual, only state transition the app performs.
Outcome	-> Real. Audit log entry written; Governance/Managed Operations reflect it immediately (no batch/reporting delay, because there is no batch layer).
Learning / Feedback	-> Not identified in the current implementation. Manager corrections update the current claim record but are not fed back into any model or threshold-tuning process. Proposed / Future Enhancement.

3. Solution Overview

Zuci Agent Services is a two-tier web application:

- **Frontend:** Angular 17.3, standalone components, TypeScript, SCSS with a centralized design-token system, Zuci's own blue/gold brand palette, light/dark themes.
- **Backend:** FastAPI (Python), organized `routers -> services -> agents/repositories`, all data in-memory, all AI calls behind one provider abstraction.

Eight routed areas: a public Landing splash (/), Login, Dashboard, Fax Intake/Claim Queue, Growth Studio, Governance, Managed Operations, plus a wildcard redirect — and a floating AI assistant available wherever a claim is open.

Two workflows exist side by side in the same running application:

1. **Claim Intake** — the real, primary product: upload -> extract -> verify -> govern -> review -> resolve. This is the backbone of the "prediction -> action" story in this document.
 2. **Growth Studio** — a secondary, consulting-style tool: describe *any* business process, get an AI-drafted (whitelist-constrained) automation assessment, simulate an agent team against a real claim, and model ROI/Revenue with plain arithmetic. This is where the clearest "AI drafts, code overrides" example lives (`agents/implementation.py`).
-

4. Application Overview

Module	Purpose	Implementation status
Landing (/)	Public "Agent Simulation" splash screen	Fully implemented
Login (mock)	Demo access gate, one hardcoded credential	Fully implemented (explicitly not production auth)
Dashboard	Entry point — the Transformation Factory funnel	Fully implemented (deliberately minimal — no analytics on this page)
Fax Intake / Claim Queue	Upload, extract, review, approve claim documents	Fully implemented
AI Claim Manager	Grounded chat about the open claim, voice I/O	Fully implemented
Growth Studio	Process assessment, agent simulation, ROI/Revenue	Fully implemented, with two of five tabs (ROI, Revenue) doing pure arithmetic, not AI
Governance	Live policy thresholds, decision-authority split, audit trail	Fully implemented (real-data view, no new AI)
Managed Operations	Live throughput/quality snapshot	Fully implemented (real-data view, no new AI)
Prediction/ML model	Forecasting, risk scoring from historical data	Not identified in the current implementation
Optimization engine	Solver comparing alternative actions	Not identified in the current implementation
Multi-role RBAC	Distinct roles with different permissions	Not identified — one mock role exists (Clinical Admin)
Backend authentication	Server-side auth on the API	Not identified — every API endpoint is unauthenticated

5. User Roles

This is the most important honesty check in the whole document: the application has exactly one role.

`Clinical Admin` is a display string set unconditionally on the single hardcoded mock login (`test1@gmail.com / 12345678` , in `frontend/src/app/services/auth.service.ts`). It is never checked against any permission table, on the frontend or the backend. `authGuard` (`frontend/src/app/guards/auth.guard.ts`) enforces only "is *someone* logged in," not "is this specific user allowed to do this specific action." Every backend API endpoint is fully open regardless of session state.

There is therefore no real Admin / Manager / Analyst distinction to document, and inventing one to fill out a role matrix would violate this document's own ground rule. The honest role matrix is:

Role	Dashboard	Claim Data	Recommendations	Approval	Configuration	Reports/Governance
Clinical Admin (only role that exists)	A	A	A	A	X (thresholds are <code>.env</code> -only, no in-app config UI)	A

A = full access, as implemented. X = no access, as implemented. No M/R/C variation exists because no second role exists to compare against.

Proposed / Future Enhancement: a real RBAC model (e.g., Reviewer vs. Approver vs. Compliance Auditor, each with different write/approve/config permissions) is a natural next step and would let a future version of this document show a genuine access matrix. Today it would be fabricated.

6. Application Modules

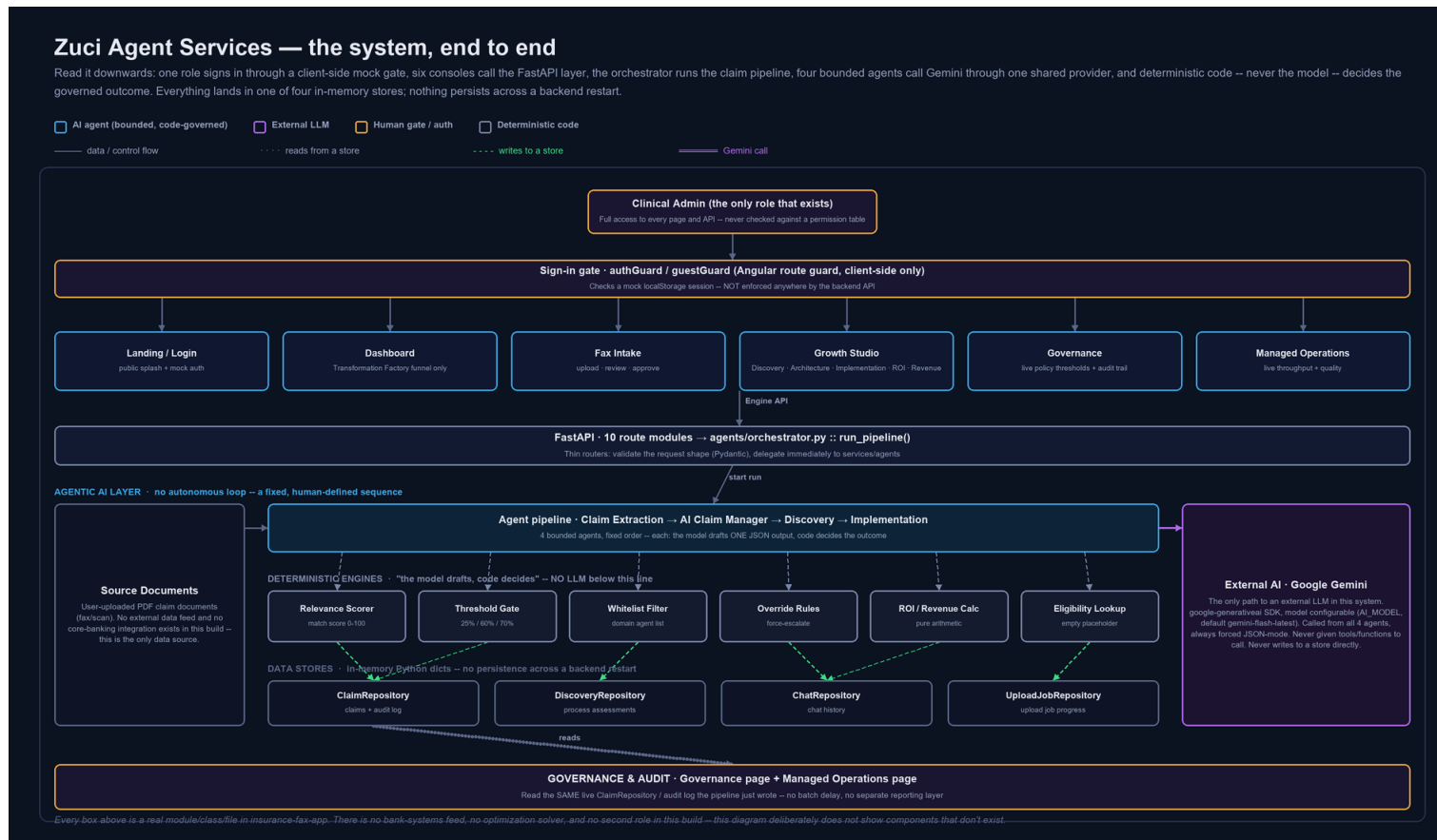
Module	Key Features	AI Usage	Status
Fax Intake pipeline	Upload, OCR fallback, extraction, verification, conflict detection	Gemini or rule-based fallback	Fully implemented
AI Claim Manager	Grounded Q&A, quick actions, voice I/O	Gemini, grounded, code-checked	Fully implemented
Growth Studio — Discovery	Process description in, suitability score + recommended agents out	Gemini, whitelist-enforced fallback	Fully implemented
Growth Studio — Implementation	Agent-team simulation against a real claim	Gemini, code-overridden	Fully implemented
Growth Studio — ROI / Revenue	Time/cost savings, commercial opportunity	None — pure arithmetic	Fully implemented, no AI involved
Governance	Live thresholds, decision split, audit trail	None — real-data view	Fully implemented
Managed Operations	Throughput, quality snapshot	None — real-data view	Fully implemented
Eligibility lookup	Member eligibility check	None — deterministic dict lookup	Configured but not usable — the lookup table (<code>ELIGIBILITY_DB</code>) is a hardcoded empty dict , so this always returns "Not Found" today

7. End-to-End Architecture

```
Landing ("/", public)
|
Login (frontend-only mock auth, no backend session)
|
Dashboard (Transformation Factory funnel)
|
Fax Intake: Upload PDF
|
PDF Extraction (PyMuPDF + pdfplumber + per-page Tesseract OCR fallback)
|
Document Relevance Scoring (deterministic keyword-signal scoring)
| (score < 25 -> HARD REJECT, no fields fabricated)
AI Field Extraction (Gemini JSON-mode, or deterministic regex fallback)
|
Source Verification (value must literally appear in the document text)
|
Cross-Page Conflict Detection (regex re-run per page)
|
Eligibility Lookup (deterministic, placeholder data source)
|
Claim Record Assembled -> status = auto_filled | needs_review | invalid
|
Human Review (Claim Queue): edit fields, view source, ask the AI Claim Manager
|
Decision: Approve & Resolve, or leave for review
|
Audit Log Entry Written
|
Governance / Managed Operations reflect the update immediately
```

Every stage above is drawn directly from `agents/orchestrator.py`'s `run_pipeline()` — see §11.3 of the companion `Application_Analysis_Hackfest_Showcase.md` for the full page-by-page trace if a line-by-line citation is needed live.

8. Architecture Diagram



Read it top to bottom, the same convention as the reference style: solid arrows are control/data flow, dotted arrows are reads, dashed arrows are writes, and the violet arrow is the only code path that ever reaches an external LLM — shared by all four agents behind one provider abstraction. Every box name above is a real module/class/file in this codebase — none are illustrative placeholders.

What is different from a typical "banking agentic AI" reference diagram, stated plainly:

- One role box, not three — this application has one mock role.
- No "bank systems" column — the only external input is a user-uploaded PDF; there is no core-banking feed.
- No optimization-engine row — the deterministic engines here are a relevance scorer, a threshold comparator, a whitelist filter, and two calculators, not a solver.
- Four in-memory Python dict stores, not SQLite/Parquet/an identity vault — nothing here persists across a backend restart.

9. Agentic AI Architecture

What "agent" means in this codebase

Every named "agent" is one of two things:

1. **A single bounded LLM call** that drafts a structured JSON output for one step, immediately followed by deterministic Python code that can override or reject that output.
2. **A purely deterministic function** with no LLM involvement (eligibility lookup, ROI/Revenue math, document/domain classification, quick-action chat answers).

There is **no autonomous loop** — no agent plans multiple steps, chooses which tool to call next, or acts without a human-defined code path dictating what happens after each model response. This should be described as **"LLM-assisted, code-governed automation,"** not autonomous agentic AI.

The four real agents

Claim Extraction Agent

- Purpose: extract 15 canonical claim fields from a document
- Input: full document text (truncated to 60,000 characters) + the 15 field names
- Output: `{value, confidence, sourceText}` per field
- Validation: every value's `sourceText` is checked to literally appear in the document; cross-page values are compared for conflicts
- Human interaction: the reviewer can edit any value; a manager-verified field is never overwritten by a later reprocess
- Fallback: deterministic regex extraction when no AI key is configured or the call fails

AI Claim Manager (grounded chat)

- Purpose: answer plain-language questions about the open claim
- Input: the question + a constructed context (extracted fields + top-3 keyword-relevant pages, "RAG-lite" — explicitly not vector-embeddings retrieval)
- Decision responsibility split: an ambiguity check and a quick-action/direct-field match are both handled **before** any LLM call, entirely in code; only a genuinely open-ended question reaches Gemini
- Output validation: the model's own JSON includes `found` and `sourceText`; if the cited `sourceText` does not literally appear in the context that was actually sent, the answer is discarded and replaced with an "unverified" message
- Tools/APIs used: none — no function/tool calling is implemented; the model is never given tools to invoke

Discovery Agent

- Purpose: assess a pasted business-process description and recommend an agent architecture
- Input: the process text + a domain-specific agent whitelist (`AGENT_WHITELIST` in `agents/discovery.py` , keyed by `healthcare` / `collections` / `customer_service` / `general`)
- Decision responsibility: domain classification is 100% deterministic keyword matching in code, **never** an LLM call — it runs *before* the prompt is built, because it gates which agents the model is even allowed to suggest
- Validation: every agent name the model returns is checked against the whitelist; anything outside it is discarded and logged (`discovery_filtered_out_of_whitelist_agents`), never shown to the user

- Fallback: a deterministic formula (`_fallback_assessment`) derived from the actual input text (step count, role keywords, action-verb keywords) — not a fixed placeholder number

Implementation / Simulation Agent

- Purpose: simulate a recommended agent team against a real, already-processed claim
- Input: the extracted claim fields (source-verified) + the recommended-agent list
- Code-level overrides that apply **regardless of what the model said**:
- if `overall_confidence < 60` , the LLM call is skipped entirely and the claim is force-escalated (`LOW_CONFIDENCE_ESCALATION_THRESHOLD = 60` in `agents/implementation.py`)
- if no diagnosis was extracted, the Policy Agent is force-flagged with an exception
- if no patient name was extracted, the final status is forced to "Human Review Required"
- This is the single clearest, most literal example in the codebase of "the model drafts, code decides."

Agent flow (generic shape, applies to all four)

```
User Request (chat question / upload / process description)
|
Context Retrieval (already-extracted fields, already-processed claim, or the pasted text itself
    -- never a second OCR/extraction pass)
|
Agent (single bounded Gemini call, JSON-mode) -- or the deterministic fallback if unavailable
|
Deterministic Engine (source-text check / whitelist filter / confidence-threshold override)
|
Recommendation / Answer (only after passing the check above)
|
Human (reviews, corrects, approves, or asks a follow-up question)
```

Responsibility split, stated once, applies everywhere in this application:

- **AI/LLM**: drafts a structured suggestion for one bounded step.
- **Deterministic code**: validates, filters, overrides, and decides the final outcome.
- **Database (in-memory)**: persists the claim/assessment/chat record and the audit trail.
- **Human**: corrects, approves, or asks a clarifying question; is the only actor who can move a claim to "Resolved."

10. LLM Integration

- **Model/provider:** Google Gemini, via the `google-generativeai` SDK. Model name is configurable (`AI_MODEL` , default `gemini-flash-latest`); the exact dated snapshot served by that alias is controlled by Google, not this codebase, and is intentionally not pinned (a pinned dated model previously broke the app when Google retired it — see the comment in `core/config.py`).
- **Where it is called:** four places — claim field extraction, the AI Claim Manager's open-ended chat branch, Discovery assessment drafting, Implementation simulation drafting. All four are forced into JSON-mode generation.
- **What is sent to the model:** document text or chat context (truncated), the 15 field names, or a process description + a domain-specific agent whitelist — always data the application already has, never a second copy fetched independently by the model.
- **What is NOT sent to the model:** login credentials, `AI_API_KEY` itself, or any data outside the currently-open claim/process. The system prompt for chat explicitly instructs the model never to reveal internal prompts or credentials (a prompt-level mitigation, not a code-level guarantee — stated as such, not oversold).
- **Response parsing/validation:** every call is wrapped in `try/except` ; a parse failure or thrown exception routes to the deterministic fallback for that feature, never a raw error surfaced to the user.
- **Token/context handling:** document/context text is truncated to 60,000 characters for extraction and to the top-3 keyword-relevant pages (each capped at 2,000 characters) for chat — an explicit, code-level cost/latency bound, not unlimited context.
- **Can the LLM directly modify business data?** No. Every LLM output lands in a Python variable that deterministic code then validates, filters, or overrides before it is ever written to the in-memory repository. The model never has a code path to write directly to the claim record.
- **Is human approval required?** Yes, for anything that does not clear the configured thresholds; "Approve & Resolve" is always a human action regardless of AI confidence.

```
Application (already-verified fields / already-classified domain / already-scored document)
|
| controlled, truncated context -- never the whole system's data
v
AI Agent (one of the four above)
|
v
LLM (Gemini, forced JSON-mode)
|
v
Code-level validation (source-text grounding / whitelist filter / confidence override)
|
v
Business Rules / Governance (the three configured thresholds; the whitelist; the override rules)
|
v
```

Action (claim status set; chat answer shown; recommendation rendered) -- only after the step above

11. Deterministic Engines

Engine	Input	Processing	Output	Why deterministic
Document Relevance Service	Raw document text	Keyword-signal scoring against four indicator groups (patient, medical, insurance, claim)	A 0–100 match score + status (invalid/review_required/valid)	Must be fast, free, and 100% reproducible so document rejection never depends on an external AI call succeeding
Confidence threshold gate	Per-field AI/rule-based confidence	Compare against <code>low_confidence_threshold = 0.70</code>	<code>auto_fill</code> or <code>needs_review</code> per field	A compliance-relevant routing decision must be auditable and cannot depend on model behavior
Discovery whitelist filter	The model's <code>recommended_agents</code> list	Keep only names present in the domain whitelist; log and discard the rest	A guaranteed-safe agent list	Prevents the model from inventing product/agent names that don't exist
Implementation override rules	<code>overall_confidence</code> , presence of diagnosis/patient name	Three hardcoded rules (skip AI below 60% confidence; force-flag missing diagnosis; force human review if no patient name)	A corrected <code>final_status</code> / <code>policy_agent</code>	Compliance-relevant escalation must not be left to the model's discretion
ROI calculator	Volume, minutes/request, automation %, cost/hour	Fixed formulas	Hours saved, annual savings	Auditable business-case math with zero LLM cost or variance
Revenue calculator	Touchpoints, agent count, domain	Fixed per-stage fee formulas, domain-weighted governance multiplier	Per-stage fee estimate + total	Same reason as ROI — a finance stakeholder needs transparent, non-AI math
Eligibility lookup	Extracted member number	Dict lookup against <code>ELIGIBILITY_DB</code>	<code>{found, client, plan, group_no, eligibility_status}</code>	Deliberately simple and auditable — this step would touch a real patient/member database in production

On the requested statement "AI recommends/explains; deterministic systems calculate, validate and govern": this is accurate for this application and is demonstrated four separate times (extraction, chat, Discovery, Implementation) — it is the single strongest, most repeatable piece of evidence in the whole codebase for the Hackfest's "agentic

prescriptive analytics, not just prediction" thesis.

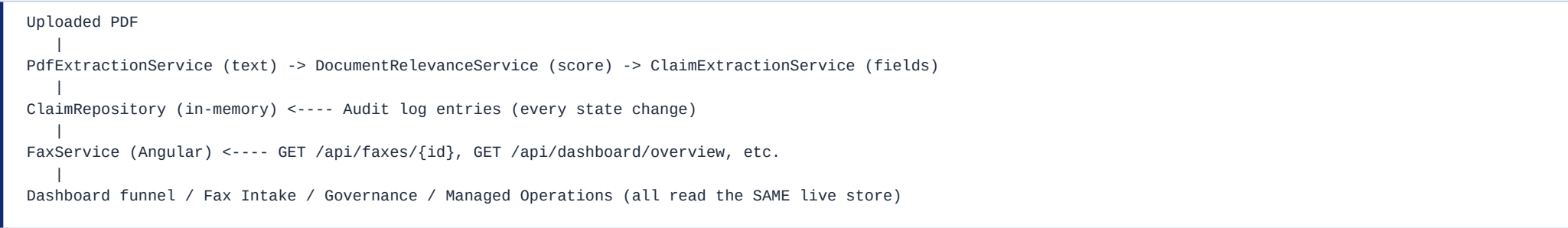
What is explicitly absent: there is no optimization solver anywhere (no MIP/CBC/linear-programming library in `requirements.txt`), no Monte Carlo/simulation engine, and no queue-prioritization engine. "Optimization" in the Growth Studio ROI/Revenue tabs means fixed-formula estimation, not comparison of alternative actions to select an optimum — this distinction should be stated plainly if a judge asks.

12. Data Architecture

Store	Technology	Purpose	Contains customer-sensitive data	Persists across restart
ClaimRepository	Python dict, in-memory	Claim records + audit log	Yes — extracted patient/claim fields	No
DiscoveryRepository	Python dict, in-memory	Stored process assessments	No	No
ChatRepository	Python dict, in-memory	AI Claim Manager conversation history	Yes — references claim data	No
UploadJobRepository	Python dict, in-memory	Async upload job progress	No	No
<code>.env</code> / <code>.env.example</code>	Flat file, git-ignored	AI provider config, thresholds, upload limits	No (secrets only)	Yes (it's a file, not app state)

Categorized:

- **Customer/claim data:** ClaimRepository (extracted patient name, DOB, diagnosis, financial amounts, etc.)
- **Operational data:** UploadJobRepository, audit log entries
- **AI/LLM data:** none stored separately — chat context is constructed per-request from the stores above, never persisted as a separate "AI dataset"
- **Configuration:** `.env` (AI provider/model/key, the three thresholds, max upload size)
- **Audit:** the audit log inside ClaimRepository — a plain list, not a tamper-evident or append-only store
- **Identity/security data:** none exists as a distinct store — the mock session lives only in the browser's `localStorage`



Not identified in the current implementation: a real database (PostgreSQL/etc.), an identity vault, a config-pack (YAML) system, or any Parquet/columnar artifact store. The repository-pattern seam (`repositories/*.py`) is the designed path to add a real database later without touching business logic — that seam exists, but no real database sits behind it today.

13. API / Backend Flow

FastAPI, one router aggregating ten route modules, every response following a consistent `{success, error:{code,message}}` shape on failure. Representative endpoints (grouped, not exhaustive — see the companion `Application_Analysis_Hackfest_Showcase.md` §28 for the complete inventory):

Endpoint	Purpose	AI involved
POST /api/faxes/upload-async	Start the async extraction pipeline	Yes
GET /api/faxes/upload-status/{jobId}	Poll real per-stage progress	No
GET /api/faxes/{id}	Full claim record	No
POST /api/faxes/{id}/decision	Human approve/reject + corrections	No
POST /api/claims/{id}/chat	Ask the grounded AI Claim Manager	Yes
POST /api/discovery/assess	Run a Discovery assessment	Yes
POST /api/implementation/simulate	Run the agent simulation	Yes
POST /api/roi/calculate	ROI math	No
GET /api/governance/policy	Live policy thresholds	No

Authentication on every endpoint above: none. This is stated plainly in §14 and is the single most important security gap to disclose to a judge who asks "how is this secured?"

Request trace, one real example (upload):

```
Browser -> POST /api/faxes/upload-async (multipart PDF)
|
FastAPI router validates request shape (Pydantic) -> delegates immediately
|
Background OS thread runs agents/orchestrator.run_pipeline()
|
```

Frontend polls GET /api/faxes/upload-status/{jobId} every 400ms for REAL stage/percent
|
On completion: claim persisted in ClaimRepository, audit entry appended
|
Browser navigates to /queue/all/{claimId}; GET /api/faxes/{id} renders the field grid

14. Security & Governance

Implemented:

- PDF upload validation (extension, content-type, non-empty, configurable max size)
- Consistent error contract; a catch-all handler prevents stack traces reaching the client
- Secrets read from `.env` (git-ignored), never hardcoded
- Logging excludes raw document text and extracted field values — metadata only (claim id, stage, duration, match score)
- Frontend route guards (`authGuard` / `guestGuard`) — client-side only
- Four separate, real AI guardrails (source-text grounding, whitelist filtering, confidence-threshold overrides — see §9/§11)

Not implemented — state these plainly:

- **No backend authentication or authorization of any kind.** Every API endpoint is fully open.
- **No RBAC** — one mock role, never checked against a permission table.
- **CORS wide open** (`allow_origins=["*"]`), explicitly a development convenience in `main.py`.
- **No rate limiting** on any endpoint, including the AI-backed ones.
- **No persistence**, so no encryption-at-rest question currently applies — but this becomes a real gap the moment persistence is added.
- **No tamper-evident audit log** — it is a plain in-memory list.

How this application prevents an AI model from directly making an uncontrolled decision: every LLM output passes through a deterministic check before it is trusted — source-text must literally appear in the document/context; recommended agents must be in the domain whitelist; low-confidence or missing-critical-field cases are force-escalated regardless of what the model proposed. The model never has a code path that writes to the claim record directly; it only ever populates a variable that governed code then validates.

15. End-to-End Business Flow

```
User (Clinical Admin, the only role)
|
Login (frontend-only mock session; role resolved to a single hardcoded string, never enforced)
|
Dashboard (Transformation Factory funnel -- no data fetching)
|
Fax Intake console -> Upload PDF
|
API: POST /api/faxes/upload-async -> Backend: agents/orchestrator.run_pipeline()
|
Prediction/Signal: document relevance score + per-field AI extraction confidence
|
AI Agent: ClaimExtractionService (Gemini or rule-based fallback)
|
Deterministic Engine: source verification, conflict detection, threshold comparison
|
Governance / Policy Validation: three configured thresholds decide reject / human-required / auto
|
Recommendation: the claim record itself, with per-field confidence and status
|
Human Approval (where the policy requires it): reviewer edits, asks the AI Claim Manager, approves
|
Action: claim status set to resolved / auto_filled / needs_review / invalid
|
Database / Audit: ClaimRepository updated, audit log entry appended
|
Frontend Response: Governance and Managed Operations reflect the change immediately
```

16. Prediction -> Action Journey

Applying the requested framework literally, honestly, to the claim-intake workflow:

Stage	Real implementation	Component
Prediction	Not identified as ML forecasting. Closest analog: relevance/match score + extraction confidence	Relevance + extraction services
Insight	Real — descriptive analytics over live claim data	Governance, Managed Operations pages
Recommended Action	Real — the claim's own routing outcome; separately, Growth Studio's Discovery recommends an agent architecture for a <i>described process</i>	Orchestrator + Discovery agent
Optimization	Not identified — no solver; ROI/Revenue are fixed-formula arithmetic	Proposed / Future Enhancement
Governance / Policy Validation	Real — three configured, code-enforced thresholds	Config + relevance service
Human Approval	Real — "Review Anyway" gate, "Approve & Resolve" action	Claim Queue UI + decision endpoint
Action	Real — claim status transition	<code>ClaimRepository</code>
Outcome	Real — audit log + live Governance/Managed Operations views	In-memory audit log
Learning / Feedback	Not identified — corrections are not fed back into any model or threshold	Proposed / Future Enhancement

17. Implemented vs Proposed Features

Capability	Status	Evidence in application	Demo usage
PDF upload + async pipeline	Implemented	Upload endpoint + orchestrator pipeline	Upload a real sample document live
OCR fallback	Implemented	PdfExtractionService + Tesseract	Upload a scanned/low-text sample
AI field extraction	Implemented	ClaimExtractionService, Gemini JSON-mode	Show the field grid populate
Deterministic fallback extraction	Implemented	RuleBasedProvider	Unset <code>AI_API_KEY</code> and re-upload live
Source verification	Implemented	<code>sourceText</code> grounding check	Click "View Source" on a field
Conflict detection	Implemented	Per-page regex re-check	Show a conflict badge on a multi-page doc
Policy-based routing (governance)	Implemented	Three thresholds in the backend config	Open the Governance page after an upload
Human approval gate	Implemented	"Approve & Resolve", "Review Anyway"	Approve a claim live
Grounded AI chat	Implemented	AI Claim Manager, code-level grounding check	Ask an open-ended question, then a quick action
Agent-name whitelist enforcement	Implemented	Discovery agent whitelist filter	Explain live: "the model can suggest, never invent"
Confidence-based simulation override	Implemented	Implementation agent override rules	Simulate a low-confidence claim, show forced escalation
Audit trail	Implemented	In-memory audit log	Show "Recent Decision Audit Trail" on Governance
ROI / Revenue calculators	Implemented (pure math)	ROI and Revenue calculation endpoints	Show the inputs/outputs are plain arithmetic

Capability	Status	Evidence in application	Demo usage
Eligibility lookup	Configured, not usable	Empty placeholder eligibility table	Show it honestly always returns "Not Found"
Multi-role RBAC	Not identified	One mock role in <code>auth.service.ts</code>	State plainly if asked
Backend authentication	Not identified	No <code>/api/auth/*</code> route exists	State plainly if asked
Prediction/ML model	Not identified	No ML dependency in <code>requirements.txt</code>	State plainly if asked
Optimization solver	Not identified	No solver dependency	State plainly if asked
Persistent database	Not identified	In-memory dicts only	State plainly if asked
Feedback/learning loop	Proposed / Future	No retraining or threshold-tuning path exists	Mention as a roadmap item
Real RBAC	Proposed / Future	—	Mention as a roadmap item

18. Hackfest Demo Flow (10–15 minutes)

#	Step	Screen	Click	Say (short)
1	Problem Statement	—	—	"Prediction-only dashboards leave the hardest part — deciding what to do — entirely to a human's unaided judgment."
2	Business Challenge	—	—	"We need the recommendation to be explainable, policy-controlled, and auditable, not just a number."
3	Solution Overview	Landing	Open the app	"This is a real, running application — everything shown is live, not slides."
4	Architecture	This document's diagram	Point top to bottom	2–3 minute walkthrough — see §20
5	Login / Role	Login	Sign in	"One role exists today by design at this stage — I'll be upfront about that."
6	Dashboard	Dashboard	Point at the funnel	"Every stage of our methodology is a real click, not a separate slide."
7	Select scenario	Fax Intake	Upload a sample PDF	"Watch the real per-stage pipeline progress."
8	Existing signal	Claim detail	Point at confidence/match score	"This is the closest thing to a 'prediction' in this build — a live, computed confidence signal."
9	Trigger agentic workflow	Claim detail	Open the AI Claim Manager	"Ask it something open-ended."

#	Step	Screen	Click	Say (short)
10	Show agent activity	Chat panel	Ask a question	"Watch which questions never touch the AI at all — that's by design."
11	Context retrieval	Chat panel	—	"The context sent to the model is the extracted fields plus the top 3 relevant pages — nothing else."
12	Recommendation	Governance	Navigate	"These are the exact thresholds that just decided this claim's routing."
13	Optimization / rule validation	Growth Studio -> Implementation	Simulate a claim	"Watch the code override the model if confidence is low or a diagnosis is missing."
14	Governance / approval	Claim detail	—	"Approve & Resolve is always a human action."
15	Execute / approve	Claim detail	Click Approve & Resolve	"That decision is now permanent and logged."
16	Show result	Governance	Refresh	"The decision-authority split just changed, live."
17	Audit / traceability	Governance	Click an audit row	"Jumps straight to the claim it references."
18	Business value	—	—	See §21
19	Future enhancements	—	—	See §23

19. Detailed Presenter Script

Opening: "What you're about to see is a real, running application — a FastAPI backend and a real Gemini call where we say so, and an equally real deterministic fallback where we say so. Nothing here is a mockup."

Framing the theme: "The Hackfest theme is prediction to action. We are not showing you a new prediction model today — we are showing the layer that most AI-in-banking pitches skip entirely: what happens *after* a signal exists. How does it become a governed, explainable, auditable action?"

On the extraction pipeline: "This document just produced two things: a relevance score and a set of field-level confidence scores. Watch what happens next — it isn't a human squinting at a number. It's a policy, enforced in code, on every document, every time."

On the AI Claim Manager: "I'm going to ask three questions on purpose. One is answered instantly with no AI call at all — pure lookup over data we already extracted. One goes to Gemini. One is deliberately ambiguous, so you can see the system ask for clarification instead of guessing."

On governance: "This page is not illustrative. These are the exact `.env`-configured numbers enforced on the document you just watched upload."

On the Implementation agent (the strongest single moment): "Watch this claim's confidence. If it's under 60%, the code skips the AI call entirely and forces escalation — the model doesn't even get asked. That's not a prompt instruction. That's a hard rule in Python."

Closing: "Everything in this build follows one repeated pattern: the model drafts, deterministic code decides. That pattern, proven four separate times in one working application, is the actual, defensible answer to 'how do you stop AI from making an ungoverned decision' — not a policy document, a running system."

20. Architecture Walkthrough Script (2–3 minutes)

Point at the diagram from §8, top to bottom, in this order:

1. "One role enters the system today — Clinical Admin. I'll say plainly this is a single-role build; a real RBAC model is future work."
 2. "The sign-in gate is a client-side mock — it protects the Angular routes, not the API. I flag that as the top security gap."
 3. "Every console you see — Fax Intake, Growth Studio, Governance, Managed Operations — calls one FastAPI layer underneath."
 4. "The API layer is thin by design — it validates the request and delegates immediately to services and agents."
 5. "The agentic layer is four bounded agents, not an autonomous loop — extraction, chat, Discovery, Implementation."
 6. "Each agent calls Gemini through one shared provider abstraction — the violet arrow on the diagram is the *only* path to an external LLM in the whole system."
 7. "Immediately after each LLM call, deterministic code validates or overrides it — source-text grounding, whitelist filtering, confidence-based escalation."
 8. "Recommendations only reach the human after that check passes."
 9. "Governance is not a separate add-on — it's the same three thresholds enforced on every document, visible live on its own page."
 10. "Human approval is the only way a claim becomes Resolved — there is no auto-approve path that skips a human when the policy requires one."
 11. "Everything lands in one of four in-memory stores — I'll be upfront that none of this persists across a backend restart today."
 12. "Every state change is logged, and that log is what Governance and Managed Operations both read from — live, no batch delay."
-

21. Business Value

- **Faster, more consistent first-pass decisions** — a policy applied identically to every document, not a judgment call that varies by reviewer.
- **From signal to action** — a confidence/relevance score doesn't just sit on a dashboard; it drives an actual routing decision, in code.
- **Explainable recommendations** — every extracted value carries its exact source snippet; every routing decision traces to a named threshold.
- **Policy-controlled decisions** — thresholds are configuration, not scattered `if` statements; changing them changes behavior everywhere at once.
- **Human-in-the-loop governance** — the system escalates rather than guesses whenever confidence is genuinely low.
- **Auditability** — every state-changing action is logged and immediately visible on a governance view.
- **Reusable architecture** — the same "model drafts, code decides" pattern is already applied four separate times in one codebase, which is evidence it generalizes rather than being a one-off trick.

Not claimed: no specific accuracy percentage, cost-reduction figure, or ROI number is asserted for this application's real-world performance — no telemetry or benchmark exists in the codebase to support one.

22. Technical Innovation

1. **The demo and the product are the same system.** Governance and Managed Operations are live views over the exact data the claim-intake pipeline just produced — not a separate slide deck.
 2. **A repeatable governance pattern, not a one-off.** The same "model drafts, code decides" shape appears at all four AI touch points, which is a stronger claim than "we added a guardrail somewhere."
 3. **Zero-dependency resilience.** Removing the AI key doesn't break the app — every AI-backed feature has a working deterministic fallback, demonstrable live.
 4. **A reusable consulting tool inside the same app.** Growth Studio accepts *any* pasted business-process description, not just insurance claims — the same governance pattern (whitelist + confidence override) applies regardless of domain.
-

23. Future Enhancements

- Add real backend authentication and a genuine multi-role RBAC model (closing the two biggest gaps identified in §5/§14).
 - Introduce an actual optimization step — comparing alternative recommended actions against constraints, not just fixed-formula estimation.
 - Introduce a genuine prediction/ML model (e.g., trained on historical claim-outcome data) feeding into the existing governance layer, rather than relying solely on point-in-time extraction confidence.
 - Add a feedback loop — manager corrections currently update only the current record; feeding them back into threshold tuning or model retraining is not implemented today.
 - Replace in-memory repositories with a real, persistent, access-controlled datastore.
 - Extend the whitelist/override governance pattern demonstrated here to additional domains beyond the current healthcare/collections/customer-service set.
-

24–25. Judge Questions and Answers

Judge question	Answer, grounded in the actual implementation
Why AI agents instead of a normal workflow?	Because the extraction and drafting steps genuinely benefit from language understanding a fixed workflow can't do (reading a scanned claim, drafting a plausible process assessment) — but every compliance-relevant decision after that draft is deterministic code, not the model.
Why not a normal deterministic pipeline for everything?	Field extraction from unstructured, scanned text is exactly the kind of task rule-based code alone handles poorly — hence the AI-first, rule-fallback design, with rules governing the outcome either way.
Why use an LLM at all?	To draft structured output from unstructured input (document text, a free-text process description) faster than hand-written parsing rules, with a deterministic fallback so the app never depends on it being available.
What does the LLM actually do?	Drafts one bounded JSON output per call — field values, a chat answer, a process assessment, or a simulated agent-team output. It never decides the final governed outcome by itself.
What remains deterministic?	Document relevance scoring, the three routing thresholds, the agent whitelist filter, the low-confidence/missing-field override rules, and both financial calculators.
How do you prevent hallucinations?	Source-text grounding (a claimed value/citation must literally appear in the document/context sent), whitelist filtering (agent names outside the list are discarded), and confidence-based escalation (low-confidence claims skip the AI call and go straight to a human).
How is customer data protected?	Logs exclude raw document text and field values (metadata only). That said, there is no backend authentication today — this should not be presented as a secured system for real PHI.
How do you handle governance?	Three configured thresholds, enforced in code on every document, visible live on the Governance page — not a policy document, a running enforcement mechanism.
How is explainability achieved?	Every extracted field carries its exact source page/snippet; every routing decision traces to a named, visible threshold.

Judge question	Answer, grounded in the actual implementation
How does human approval work?	Borderline documents require an explicit "Review Anyway"; every claim's final "Resolved" state requires an explicit "Approve & Resolve" click — there is no fully automatic path to Resolved.
How does the system scale?	Not proven today — single-process, in-memory, no distributed task queue. The repository pattern is the designed seam for a future database swap.
How do you measure recommendation quality?	Not identified in the current implementation — no telemetry or benchmark exists to score recommendation accuracy against outcomes.
How do you prevent unauthorized actions?	Not fully solved today — the API has no backend authentication; this is disclosed as the top security gap, not hidden.
What happens if the AI fails?	Every Gemini call is wrapped in <code>try/except</code> ; a failure falls back to the deterministic path for that feature (rule-based extraction, fallback assessment/simulation, or a clear "AI unavailable" chat message) — the workflow never crashes.
What happens if an API fails?	A consistent typed-exception -> JSON error contract; a catch-all handler returns a generic 500 without leaking internals.
How does this integrate with existing banking prediction systems?	Not identified in the current implementation — there is no external prediction-system integration point today; the closest analog is the app's own extraction-confidence signal.
What is innovative here?	The same governed "AI drafts, code decides" pattern repeated at four independent points in one working system, plus a live governance page reading the same data the pipeline just produced.
How can this move from demo to production?	Add backend auth + RBAC, swap in a real database behind the existing repository seam, add rate limiting, and connect a real eligibility/prediction data source — none of these require a rewrite of the agent/governance pattern already in place.

26. One-Minute Elevator Pitch

"Most AI-in-banking demos stop at a prediction. We're showing what comes after: a real, running application where a document's own confidence signal drives a policy-governed decision — auto-process, human-required, or reject — enforced by code, not judgment. Every AI recommendation in this system is drafted by a model and then checked, filtered, or overridden by deterministic rules before a human ever sees it, and every decision is logged and explainable. That governance pattern — the model drafts, the code decides — repeats four separate times in one codebase, which is why we can say it generalizes rather than being a one-off trick."

27. Three-Minute Technical Pitch

"This is Zuci Agent Services — an Angular and FastAPI application with one job at its core: take an unstructured insurance claim document and turn it into a structured, source-verified, policy-routed record, with a human kept in the loop exactly where the system isn't confident enough to act alone.

The pipeline is five real stages: PDF extraction with an automatic OCR fallback, a deterministic relevance scorer that rejects documents outright below a configured match threshold, AI-assisted field extraction with an automatic rule-based fallback if no AI key is configured, a code-level source-verification check on every extracted value, and a per-page conflict-detection pass.

The output of that pipeline — a match score and per-field confidence — is the closest thing to a 'prediction' in this build, and I'll say plainly it is not a trained ML model; it's a live, computed signal. What happens to that signal is the real story: three configured thresholds, enforced in Python, decide whether the claim auto-processes, needs a human, or gets rejected — the same policy, every time, visible live on a dedicated Governance page reading the exact same claim data.

The same pattern — model drafts, code decides — repeats in three more places: the AI Claim Manager's chat answers are discarded unless their cited source text is actually verified in the context sent; the Discovery agent's recommended automation agents are filtered against a domain whitelist in code; and the Implementation agent's simulated outcome is force-overridden if confidence is low or a critical field is missing, regardless of what the model proposed.

What I won't claim: there's no trained prediction model, no optimization solver, no multi-role access control, and no backend authentication today. Those are real gaps, not hidden ones, and they define the honest next steps rather than undermining what's already proven to work."

28. Final Demo Checklist

- [] Backend running (`uvicorn`) and confirmed healthy via `/api/health`
 - [] Frontend running (`ng serve`) and logged in with the demo credential
 - [] A clean sample PDF ready for the live upload (`sample-documents/professional_insurance_claim.pdf`)
 - [] A second, low-confidence or irrelevant sample ready to show the reject/escalation paths live
 - [] `AI_API_KEY` confirmed configured (to show the real Gemini path) — and know how to unset it live to show the deterministic fallback on request
 - [] Governance page pre-verified to show non-zero decision-authority data (upload at least one claim beforehand if the backend was just restarted, since nothing persists)
 - [] Growth Studio: a process description ready to paste for a live Discovery run
 - [] Know the three threshold numbers by heart: 25% (reject), 60% (review), 70% (per-field confidence)
 - [] Rehearsed the honest gaps list (§17) so a judge question never gets an evasive answer
 - [] This document's architecture diagram open/printed for the walkthrough in §20
-

Hackfest Story (closing)

Zuci Agent Services doesn't claim to predict anything — it claims something more specific and more provable: that a signal a system already has (a confidence score, a relevance score) can be turned into a governed, explainable, auditable action without letting an AI model make the final call alone. Four times over, in four different features, the same pattern holds: a model drafts a suggestion, and deterministic code — a threshold, a whitelist, an override rule — decides what actually happens, with a human as the last word whenever the policy says so. That is the whole Hackfest thesis, proven on a working application rather than argued in slides — and the honest list of what isn't built yet (prediction modeling, optimization, real RBAC, backend auth) is exactly the roadmap for turning this from a strong demo into a production-grade agentic decisioning platform.